
SWHV CCNx JHV Heliocentric 3D data interface

SWHV-ROB-IF-001-CCNx v0.9

SWHV Team



SWHV-ROB-IF-001-CCNx - Version 0.9 - 2026-09-03

Contents

1	Introduction	1
2	SunJSON	2
2.1	File structure	2
2.2	Geometry and appearance	2
2.3	Loading and time selection	3
3	glTF	4
3.1	glTF and GLB	4
3.2	Heliocentric Cartesian coordinates	4
3.3	Scene metadata	5
3.4	Supported glTF content	6
3.5	COCONUT example	7
3.5.1	Overview	7
3.5.2	Running the converter	8
3.5.3	Assumptions and processing	8
3.5.4	Output and validation	9
4	WCS	10
4.1	Heliocentric Cartesian frame	10
4.2	Regular-grid WCS	10
4.2.1	Axes and units	11
4.2.2	Reference position and linear mapping	11
5	FITS	13
5.1	File and array organization	13
5.2	Scalar encoding and units	13
5.3	Undefined voxels	14
5.4	Time and provenance	14
5.5	FITS header example	15
5.6	Lossless tiled compression	15
6	References	17
7	Changelog	18
7.1	Changes in version 0.9	18

1 Introduction

id: b1acfc17ad32747ba06bbda86e83d70c7ac2cc34

Table 1.1: Document history

Date	Notes
2026-08-19	Initial issue (unnumbered)
2026-09-03	Version 0.9

This document defines two ways to supply heliocentric 3D geometry to JHelioviewer (JHV): SunJSON and glTF 2.0. Each product represents selected model quantities through geometry and colors, and JHV places and renders it alongside its other solar data layers.

SunJSON is a small, JHV-specific JSON format for points, connected lines, and ellipses in Carrington spherical coordinates. It also records an observation time and JHV-specific display sizes. SunJSON was introduced to display model output throughout the heliosphere and is used by Qorona for its JHV field-line products.

glTF is a general 3D asset standard for products that need triangle surfaces, textures, materials, a static object hierarchy, or a combination of surfaces, lines, and points. Its Cartesian geometry can be displayed in general glTF viewers. To place that geometry correctly alongside solar data, JHV needs additional metadata describing its world coordinate system (WCS), which relates the Cartesian positions to the Sun. The glTF asset stores this metadata as keyword–value pairs in its scene JSON.

The glTF section illustrates the conversion process with COCONUT output, and the interfaces also accommodate heliocentric products derived from other models.

For products made only from points, lines, and ellipses, SunJSON is usually the more direct route. glTF is more suitable for meshed surfaces or when portability outside JHV matters.

The WCS section explains how the coordinate keywords describe a common solar frame and how additional keywords locate samples on a regular grid. The FITS section then describes storage of those samples, including scalar encoding, undefined values, and compression. JHV does not currently load or render FITS volumes.

2 SunJSON

The SunJSON format definition presented here was extracted from older project documents to consolidate JHV's full heliocentric 3D data interface in one document.

SunJSON draws on GeoJSON's simple organization of geometry but defines its own heliocentric coordinates and fields. Each file contains one timestamp and a list of geometry entries:

```
{
  "type": "SunJSON",
  "time": "2025-10-09T18:19:52.000",
  "geometry": [
    {
      "type": "line",
      "coordinates": [[1.1, 135, 45], [1.2, 135, 45], [1.3, 135, 45]],
      "colors": [[255, 255, 255, 255], [255, 0, 0, 255], [0, 0, 255, 255]],
      "thickness": 0.016
    },
    {
      "type": "point",
      "coordinates": [[1.1, 45, 45], [1.3, 45, 45]],
      "colors": [[255, 180, 0, 255]],
      "thickness": 0.01
    },
    {
      "type": "ellipse",
      "coordinates": [[2, 60, 0], [1, 60, 90], [3, 60, 0]],
      "colors": [[255, 0, 0, 192]],
      "thickness": 0.004
    }
  ]
}
```

2.1 File structure

The top-level fields are:

- **type**: required, with the value `SunJSON`;
- **time**: required timestamp in the form `YYYY-MM-DDTHH:mm:ss`, optionally followed by fractional seconds. JHV assumes UTC and does not accept timezone designators or numeric offsets; and
- **geometry**: required array of geometry entries. It may contain points, lines, and ellipses in any order.

2.2 Geometry and appearance

Every geometry entry has four fields:

- **type**: exactly `point`, `line`, or `ellipse`;
- **coordinates**: an array of finite three-component coordinates;
- **colors**: an array of four-component colors; and
- **thickness**: a finite number controlling the rendered line thickness or point size.

Each coordinate is [`radius`, `Carrington longitude`, `Carrington latitude`], where `radius` is the heliocentric distance in solar radii and both angles are in degrees. JHV converts these spherical coordinates directly into the Carrington frame without WCS metadata, since the coordinate system is fixed by the SunJSON format. A radius below one solar radius is accepted but produces a warning because it places the coordinate beneath the nominal solar surface.

The geometry types interpret their coordinates as follows:

- A `point` entry needs at least one coordinate and produces one point for each coordinate.
- A `line` entry needs at least two coordinates. JHV joins consecutive coordinates into one connected polyline.
- An `ellipse` entry needs exactly three coordinates. After conversion to Cartesian points, let them be C , U , and V . JHV draws the closed curve $C + (U - C) \cos(t) + (V - C) \sin(t)$ for $0 \leq t \leq 2\pi$. Here, C is the center, and the offsets $U - C$ and $V - C$ define the curve's two Cartesian directions and lengths.

Colors are specified as straight, non-premultiplied RGBA values, with integer components from 0 through 255. JHV clamps values outside that range and converts the result to its premultiplied representation. When fewer colors than coordinates are supplied, the last color is repeated. Extra colors are ignored, and an ellipse uses only its first color. The `colors` field must be present, although an empty array may be used to select JHV's default green.

`thickness` is a JHV display parameter, not a physical width measured in solar radii. For a `point` entry, it controls point size instead of line thickness. Its mapping to screen width or point size is implementation-dependent. Visualizing the data in JHV helps assess whether the line width or point size is suitable.

2.3 Loading and time selection

JHV loads SunJSON from local files and HTTP URIs. Dragging a local `.json` file or an HTTP URI ending in `.json` into JHV selects the SunJSON loader. A SAMP client can instead send `jhv.load.sunjson` with either a `url` parameter or a `value` parameter containing the complete JSON text. The data is loaded into the Connection layer and drawn only when that layer is enabled.

When several files are loaded, JHV treats them as a time sequence and displays the geometry whose timestamp is nearest to the current JHV time.

3 glTF

glTF 2.0 is an open standard maintained by Khronos for exchanging 3D assets. A glTF asset can combine geometry, colors, materials, textures, and a hierarchy of objects in a form understood by many 3D tools and viewers.

JHV supports the subset defined in this section. Heliocentric products add scene metadata so that JHV can orient and place their Cartesian geometry in the Carrington frame.

3.1 glTF and GLB

glTF has two file forms:

- `.gltf` stores the scene description as JSON and may refer to separate binary buffers and images or embed them as Base64 data;
- `.glb` can package the JSON and binary resources in one file.

GLB is convenient for distribution and avoids Base64 expansion. Its binary payload is about 25% smaller than the same payload embedded as Base64, but packaging external binary resources into GLB does not itself reduce their size.

Either form can be gzip-compressed without altering its content. JHV accepts `.gltf.gz` and `.glb.gz`. An HTTP server may instead use `Content-Encoding: gzip` while transferring the unmodified asset. Compression complements rather than replaces careful geometry simplification or decimation where accuracy permits. Reducing the geometry also lowers memory use after loading and the amount of work needed to render the asset.

3.2 Heliocentric Cartesian coordinates

A glTF `POSITION` attribute always contains Cartesian `x`, `y`, and `z` components. General glTF viewers interpret spherical tuples such as [`radius`, `longitude`, `latitude`] as Cartesian coordinates and place the vertices incorrectly.

Positions are measured in solar radii from the center of the Sun. Their `x`, `y`, and `z` components follow the heliocentric Cartesian convention: `SOLX` points toward solar west and `SOLY` toward solar north in the observer's image plane, and `SOLZ` points toward the observer. JHV requires the following coordinate declaration:

<code>CTYPE1 = SOLX</code>	<code>CUNIT1 = solRad</code>
<code>CTYPE2 = SOLY</code>	<code>CUNIT2 = solRad</code>

```
CTYPE3 = SOLZ    CUNIT3 = solRad
RSUN_REF = 695700000.0
WCSNAME = Heliocentric-cartesian
```

JHV uses the `CRLN_OBS` and `CRLT_OBS` values in the **scene metadata** to rotate these positions into its Carrington world frame. This interface uses a simplified subset of FITS WCS metadata for glTF, limited to the coordinate frame, units, and orientation. glTF supplies explicit vertex positions and node transforms, so it needs no mapping from array indices and does not use `CRPIX`, `CRVAL`, `CDELTA`, or `PC/CD` matrices to place those vertices. The **WCS** section describes the shared coordinate frame and the additional mapping needed for regular grids.

3.3 Scene metadata

glTF does not define solar coordinates or observation frames. This interface uses FITS-style keyword-value pairs for the solar metadata, stored as JSON properties in `extras` on the default glTF scene:

```
{
  "scene": 0,
  "scenes": [
    {
      "name": "coronal model",
      "extras": {
        "DATE-OBS": "2025-10-09T18:19:52.000",
        "DSUN_OBS": 149597870700.0,
        "CRLN_OBS": 0.0,
        "CRLT_OBS": 0.0,
        "RSUN_REF": 695700000.0,
        "CTYPE1": "SOLX", "CUNIT1": "solRad",
        "CTYPE2": "SOLY", "CUNIT2": "solRad",
        "CTYPE3": "SOLZ", "CUNIT3": "solRad",
        "WCSNAME": "Heliocentric-cartesian"
      }
    }
  ]
}
```

`DATE-OBS` is the observation or model-state time in the form `YYYY-MM-DDTHH:mm:ss`, optionally followed by fractional seconds. JHV assumes UTC and does not accept timezone designators or numeric offsets. Every heliocentric glTF product must include `DATE-OBS`, which identifies its place in a model time sequence.

`CRLN_OBS` and `CRLT_OBS` give, in degrees, the Carrington longitude and latitude of the `SOLZ` direction at `DATE-OBS`. Together they define the orientation of all three Cartesian axes. They may describe a physical observer or a reference direction chosen for a model product. In either case, the positions must be expressed in the corresponding `SOLX`, `SOLY`, and `SOLZ` frame. Latitude must be between -90 and 90 degrees.

`DSUN_OBS` gives the distance of the observer or reference point from Sun center in meters. A model product using only a reference direction may omit it or supply a conventional positive distance. JHV validates the value when it is present but does not currently use it to place the geometry.

The presence of `WCSNAME` identifies a JHV heliocentric declaration, which must be complete and valid. JHV requires `DATE-OBS`, `CTYPE1` through `CTYPE3`, `CUNIT1` through `CUNIT3`, `RSUN_REF`, `CRLN_OBS`,

and `CRLT_OBS`, and also validates `DSUN_OBS` if supplied. Numeric values must be finite JSON numbers, not quoted strings.

Without `WCSNAME`, JHV does not interpret these fields as solar metadata. It treats the asset's positions as world coordinates and assigns the application-start time, as it does for an image without metadata. This allows ordinary glTF assets to open but does not provide the placement and time required by this heliocentric interface.

JHV applies this placement to triangles, lines, and points. Rotating the JHV view changes the camera, not the product's coordinates. Other glTF viewers do not interpret the solar metadata and display the Cartesian geometry without Carrington placement.

3.4 Supported glTF content

JHV uses Assimp, the Open Asset Import Library, to read both glTF file forms.

Geometry and scene structure. JHV renders open or closed triangle surfaces, connected lines and poly-lines, and point sets. It follows the node hierarchy and applies each object's translation, rotation, and scale before solar placement. The same mesh may be used by several nodes. Animations, skeletal deformation (skins), and morph targets are not supported.

Because glTF does not define a portable line width or point size, JHV uses fixed values for both. SunJSON is the better choice when a JHV-specific product needs to control these display sizes.

Colors, materials, and textures. Triangles, lines, and points may use a material base color and per-vertex RGB or RGBA colors. Color components are interpreted in the range 0 to 1, including vertex colors stored as normalized unsigned integers, and RGB colors have an implicit alpha of 1. Triangle materials may also use one base-color texture, embedded in the asset or referenced as an external image. JHV multiplies these color contributions to obtain the displayed color. Textures are not supported on lines or points.

glTF colors use straight alpha and are stored as non-premultiplied values, which JHV converts to the premultiplied representation used by its renderer. JHV supports opaque (`OPAQUE`), cut-out (`MASK`), and translucent (`BLEND`) materials. On triangles, `MASK` applies the material's alpha cutoff after interpolating colors and sampling any texture. On lines and points, JHV applies the cutoff to vertex colors before rendering, so transitions along a line segment are only approximate. Additive blending, separate opacity textures, and transformed texture coordinates are not supported.

Lighting and surfaces. The Khronos glTF extension `KHR_materials_unlit` tells renderers to display a material without lighting. JHV follows this rule for triangle materials and lights other triangle materials using the supplied vertex normals. JHV's simple viewer-facing light leaves a surface facing the viewer at full brightness and darkens surfaces angled away, down to a 30 percent ambient contribution. The light follows the view as the scene is rotated. Shading helps reveal surface shape, but it also changes the brightness of the colors used to encode data values. Unlit materials keep those colors independent of lighting. JHV always draws lines and points without lighting.

Lit triangle meshes must include vertex normals that describe the intended surface. JHV does not generate missing normals because doing so could smooth edges that were meant to remain sharp. Unlit

triangle meshes do not need normals, and JHV discards them if they are present. Omitting them from an unlit product also reduces its file size and memory use. JHV does not implement the complete glTF metallic/roughness model, normal maps, emissive materials, or lights and cameras stored in the asset.

JHV respects single- and double-sided triangle materials. A double-sided surface is visible from either side, but a single-sided surface disappears when viewed from its back. Opaque surfaces hide geometry behind them, including geometry from other JHV layers.

Translucent surfaces use alpha blending, whose result depends on drawing order. JHV sorts translucent mesh primitives back to front by their centers rather than sorting individual triangles. Storing spatially separate translucent components as separate glTF mesh primitives allows JHV to order them independently. Intersecting surfaces and self-overlap within a mesh can still produce incorrect blending.

Preparing a product. For products that use color to show a physical quantity, scene [extras](#) describes the quantity, units, value range, and color map. It can also record the source data, software versions, and processing parameters used to generate the product.

Careful geometry simplification can substantially decrease file size, transfer time, and rendering cost. Dense lines can be simplified and triangle meshes decimated using an error criterion appropriate to the quantity and structures being shown. Comparing the reduced product with the full result helps assess changes to important boundaries, shapes, and data values. The reduction criterion can be recorded with the other processing parameters.

Viewing the product from several directions in JHV helps assess its placement, surface orientation, and transparency alongside the solar sphere and other layers. An independent glTF viewer provides a useful comparison and can reveal portability problems.

This interface covers the products considered so far. As other model outputs become available, they may need ways to represent or display data that are not covered here. JHV's glTF support can be expanded where feasible to accommodate those uses.

3.5 COCONUT example

3.5.1 Overview

[extra/test/create_coconut_scene.py](#) shows how Qorona, PyVista/VTK, and pygltflib can be combined to produce a glTF asset using the features JHV supports. It is a starting point for model-specific converters, not a general COCONUT exporter, and is tailored to the supplied COCONUT sample CFmesh. The resulting `coconut-corona-scene.glb` contains:

- unlit magnetic field lines colored by polarity;
- an unlit triangulated $B_r=0$ current-sheet surface colored by radial plasma velocity;
- unlit point markers at the inner and outer boundary endpoints of complete open field lines; and
- eleven selected closed field lines represented by lit, thick yellow tubes with smooth vertex normals.

The tube centerlines follow field lines traced from the supplied background magnetic field. Their selection, radius, color, and representation as solid tubes are illustrative choices, not physical properties of the modeled structures. They demonstrate a construction that could be adapted to visualize model-derived flux ropes.

The current sheet is unlit because its color map represents radial plasma velocity. Shading it would change the brightness according to surface orientation, making the same velocity appear as different colors across the mesh. The field-line and boundary-point materials are also marked as unlit so that general glTF viewers preserve their polarity colors. JHV does not shade line or point primitives even without this extension. The tubes, in contrast, have a constant illustrative color and remain lit so that their round cross-section and three-dimensional shape are visible.

Qorona can already export field lines as SunJSON. This example uses glTF to combine lines, surfaces, and points in one asset that general glTF viewers can also display.

3.5.2 Running the converter

The following command runs the converter from the repository root in an environment containing Qorona 0.4.0, PyVista/VTK, Matplotlib, and pygltlib:

```
python extra/test/create_coconut_scene.py \
/path/to/coconut_corona.CFmesh.xz \
--timestamp 2025-10-09T18:19:52 \
--output extra/test/data/coconut-corona-scene.glb
```

3.5.3 Assumptions and processing

The CFmesh file does not identify its observation time or coordinate frame. For a reproducible conversion, the script uses the following assumptions and settings:

- for this CFmesh, the Qorona Cartesian coordinates are assumed to be Carrington-aligned, with +x at Carrington longitude zero, +y at Carrington longitude 90 degrees, and +z toward solar north;
- the output uses the reference direction `CRLN_OBS = 0`, `CRLT_OBS = 0` and records a conventional `DSUN_OBS` of 1 au. The converter therefore writes `(SOLX, SOLY, SOLZ) = (y, z, x)`. This fixed axis reordering preserves the model's Carrington orientation and does not depend on Earth's position at the supplied time;
- the magnetic field and velocity from the native cells are resampled with Qorona's degree-1 moving-least-squares method onto a `192 x 180 x 360` spherical grid, with logarithmic radial spacing from 1 to 6 solar radii;
- field lines are traced in float64 with DOPRI5, `rtol=10^-8`, `cfl=0.125`, and an `18 x 36` seed grid. Their glTF positions are stored as float32;
- after tracing, the field-line paths are simplified with the Ramer-Douglas-Peucker algorithm so that no removed trace vertex is farther than 10^{-5} solar radii (about 7 km for the adopted solar radius) from the simplified path. The same step is applied to the centerlines used to make the tubes in this example;
- both boundary endpoints of each complete open field line are exported as points, with the line's polarity color;

- eleven additional closed field lines are traced from equally spaced seeds at Carrington latitude 6 degrees and longitudes 32 through 42 degrees, then converted to 4 Mm-radius tubes with 16 sides;
- the current sheet is extracted as the $B_r=0$ isosurface of the reconstructed magnetic field, matching the geometric definition used by Guo et al. (2024, Fig. 1d), with the periodic longitude seam closed before meshing;
- the model velocity is interpolated at each surface vertex, converted with the COOLFluid `corona` normalization $v_0 = 480 \text{ km/s}$, and projected onto the radial direction at that vertex;
- the extracted current-sheet mesh undergoes scalar-aware quadric decimation with a target reduction of 50 percent, using radial velocity in the decimation error metric. Vertex colors are generated from the resulting scalar values after this step; and
- radial velocity is mapped through the `turbo` color map over -30 to 300 km/s, with a common surface alpha of 0.35.

3.5.4 Output and validation

The default scene's `extras` object records the source name and SHA-256 digest, Qorona version, processing parameters, surface definition, velocity mapping, and geometry counts before and after polyline simplification and current-sheet decimation. These operations reduce file size without reducing the reconstruction-grid resolution or changing the field-line tracing tolerances.

The script uses PyVista and VTK to create the geometry and obtain a glTF document in memory. It adds the solar metadata and uses pygltflib to package the document directly as GLB. Colors are stored as straight RGBA values in normalized unsigned-byte attributes, and the current sheet is double-sided and alpha-blended. The field-line, current-sheet, and point materials are marked with `KHR_materials_unlit`. The tubes remain lit and carry one smooth normal per vertex.

Before export, the converter checks the geometry and color arrays, the tube normals, and the boundary-point positions and polarity colors. It also removes adjacent line vertices that become identical after float32 conversion.

After writing, it reopens the GLB and checks the default scene and its metadata, the embedded binary buffer and its length, the expected geometry types, and the declared attribute types and counts. It verifies the lit and unlit materials and the current sheet's blending and double-sided settings. These checks do not compare every exported coordinate, color, normal, or index with the input arrays.

4 WCS

A WCS description relates a product's coordinates to physical positions through a set of keyword-value pairs. These identify the coordinate axes, units, and orientation and, for a regular grid, define how array indices map to positions in that frame. The keywords can be stored as JSON properties in glTF's scene `extras` or as records in a FITS header. The definitions below follow FITS WCS conventions for heliocentric Cartesian coordinates.

4.1 Heliocentric Cartesian frame

The coordinate axes follow the heliocentric Cartesian convention described by Thompson (2006, Section 3.1):

- `SOLX` points toward solar west in the observer's image plane;
- `SOLY` points toward solar north in the observer's image plane;
- `SOLZ` points from Sun center toward the observer.

The coordinate origin is the center of the Sun. `CRLN_OBS` and `CRLT_OBS` give the Carrington direction of the `SOLZ` axis in degrees. A physical observation uses the observer's direction. A model product that is not tied to a viewpoint may instead use a reference direction matching its native axes. At `CRLN_OBS` = 0 and `CRLT_OBS` = 0, `SOLZ` points toward Carrington longitude zero in the solar equatorial plane, `SOLX` points toward Carrington longitude 90 degrees, and `SOLY` points north. Other products may declare a non-zero direction. For glTF geometry, JHV uses the declared direction to rotate the coordinates into its Carrington world frame.

The coordinate-system name is `WCSNAME` = `'Heliocentric-cartesian'`. The adopted physical solar radius is `RSUN_REF` = 695700000.0 meters, so positions expressed in solar radii have a common scale. The glTF section specifies how to record these values and the product time in scene metadata.

`DSUN_OBS`, when supplied, records the distance from Sun center to the observer or chosen reference point in meters. It does not move the coordinate origin away from Sun center. For a physical observer, its direction and distance correspond to the product time.

4.2 Regular-grid WCS

A regular three-dimensional grid uses a linear mapping, with a reference position, to locate its voxel centers in the heliocentric Cartesian frame. This is an affine WCS, expressed through the following FITS keywords.

An unstructured or curvilinear source can be represented here by a scalar quantity resampled onto a regular grid. The sampling method and resolution depend on the quantity and the spatial structures being represented.

4.2.1 Axes and units

`CTYPE1` . . 3 identify the world-coordinate components and contain `SOLX`, `SOLY`, and `SOLZ` exactly once. The WCS matrix establishes how a step along each pixel axis changes those world coordinates, so the pixel axes need not be aligned with the world axes. Using `SOLX`, `SOLY`, and `SOLZ` in that order is nevertheless the easiest arrangement to inspect when no permutation is needed.

Each world axis has a length unit, specified by one of the following `CUNITi` values:

Unit	Interpretation
<code>solRad</code>	solar radii, using 695700000 meters per radius
<code>m</code>	meters
<code>km</code>	kilometers
<code>Mm</code>	megameters

For glTF products, this interface requires solar radii and the fixed component order given in the **coordinate declaration**.

4.2.2 Reference position and linear mapping

FITS assigns coordinate 1 to the center of the first pixel or voxel. `CRPIX` identifies a reference point in these pixel coordinates, and `CRVAL` gives the heliocentric Cartesian coordinates of the same point. The array center is a convenient reference position, given along each axis by:

$$\text{CRPIX}_i = (\text{NAXIS}_i + 1) / 2$$

With this choice, `CRVAL` gives the array center's heliocentric position: zero for a Sun-centered grid, or non-zero values describing an offset grid.

For any pixel-coordinate vector `p`, the matrix `M` converts its displacement from `CRPIX` into a heliocentric Cartesian displacement, giving:

$$\text{world} = \text{CRVAL} + \text{M} (\text{p} - \text{CRPIX})$$

The matrix has three possible representations:

- With only `CDELTi`, `M` is diagonal: $\text{M}_{i,i} = \text{CDELT}_i$.
- With a PC matrix, omitted diagonal elements are 1 and all other omitted elements are 0. Then $\text{M}_{i,j} = \text{CDELT}_i * \text{PC}_{i,j}$.
- With a CD matrix, $\text{M}_{i,j} = \text{CD}_{i,j}$. An omitted CD element is 0, and `CDELTi` is not used.

Each element $M_{i,j}$ gives the change in world coordinate i for a one-pixel step along array axis j , expressed in $CUNIT_i$. PC and CD are alternative representations and are not combined. The mapping requires a finite, nonsingular matrix and can include rotations, reflections, axis permutations, unequal scales, or linear shear.

For cell-centered voxels, the volume extends half a pixel step beyond the first and last sample centers. Its outer boundaries therefore lie at pixel coordinates 0.5 and $NAXIS_i + 0.5$ along each axis, mapped into heliocentric coordinates by the same formula.

5 FITS

The Flexible Image Transport System (FITS) stores grid samples in an array and the WCS keyword–value pairs alongside the array dimensions in its header. The **regular-grid WCS** defines where the samples are located, independently of their values and of any compression used to store them. The storage conventions below are useful for exchanging such grids, although FITS volume loading and rendering remain outside the current JHV interface.

5.1 File and array organization

A FITS file consists of header/data units (HDUs). An uncompressed cube can occupy the primary HDU or an **IMAGE** extension. If several quantities are stored in one file, their image extensions can be distinguished by **EXTNAME**, each with its own spatial and scalar metadata.

NAXIS=3 and positive **NAXIS1** . . 3 give the dimensions of a three-dimensional array. FITS axis 1 is the fastest-varying dimension. With zero-based array indices **x**, **y**, and **z**, the linear sample index is:

$$x + \text{NAXIS1} * (y + \text{NAXIS2} * z)$$

These are array indices, not solar coordinates. Adding one to each index gives the one-based FITS pixel coordinates used in the WCS mapping. In NumPy and Astropy, the corresponding array shape is (**NAXIS3**, **NAXIS2**, **NAXIS1**), and a sample is accessed as `data[z, y, x]`.

5.2 Scalar encoding and units

BITPIX identifies the stored sample type. FITS defines the following types, independently of any application's display capabilities:

BITPIX	Stored sample type
8	unsigned 8-bit integer
16	signed 16-bit integer
32	signed 32-bit integer
64	signed 64-bit integer
-32	IEEE 32-bit floating point
-64	IEEE 64-bit floating point

BSCALE and **BZERO** relate a stored sample s to the scalar value q it represents:

$$q = \text{BZERO} + \text{BSCALE} * s$$

Their defaults are **BSCALE**=1 and **BZERO**=0. This linear encoding allows integer samples to represent fractional values or values outside the stored integer range. For example, **BITPIX**=16, **BSCALE**=1, and **BZERO**=32768 represent unsigned values from 0 through 65535 using stored integers from -32768 through 32767.

Any normalization, clipping, or nonlinear transform applied before encoding forms part of the stored quantity's definition. **BSCALE** and **BZERO** describe only its linear encoding, not the preceding processing.

BUNIT gives the unit of q , after scaling, and may be omitted for a dimensionless quantity. For example, electron density in inverse cubic meters has **BUNIT**='m⁻³'.

5.3 Undefined voxels

Undefined voxels can represent regions outside the model domain, missing input data, or interpolation failures. A low or zero value is not, by itself, a reason to mark a voxel as undefined.

For integer arrays, **BLANK** reserves one stored integer to identify undefined samples. It is compared with the stored values before applying **BSCALE** and **BZERO**, and must be within the stored range. For example:

- **BITPIX**=8: 0 through 255;
- **BITPIX**=16: -32768 through 32767.

The reserved code is unavailable for valid samples, including values produced by rounding or clipping. With **BITPIX**=8 and **BLANK**=255, for example, codes 0 through 254 remain available for valid samples. When all voxels are defined, omitting **BLANK** leaves every code available for data.

Floating-point FITS arrays use NaN for undefined values. The **BLANK** keyword is prohibited in floating-point image HDUs (**BITPIX**=-32 or **BITPIX**=-64).

5.4 Time and provenance

For an instantaneous model state, **DATE-OBS** records its time. For a product covering an interval, **DATE-AVG** can record its average time. These timestamps have the form **YYYY-MM-DDTHH:mm:ss**, optionally followed by fractional seconds, and do not include timezone designators or numeric offsets. The time scale is recorded in **TIMESYS**, whose default is **UTC**.

OBJECT identifies the observed or modeled target, and **EXTNAME** can name an image extension. A descriptive **BTYPE** can identify the stored quantity. **HISTORY** records its derivation, including source data, software versions, resampling, normalization, clipping, and quantization where relevant. Source identifiers or checksums help identify the exact inputs used to produce the file.

5.5 FITS header example

This header excerpt describes a $256 \times 256 \times 256$, Sun-centered volume extending from -3 to +3 solar radii along each observer-aligned axis. Stored codes 0 through 254 represent a dimensionless scalar from 0 to 1, and 255 is reserved for undefined voxels. The scalar range is illustrative, not a prescribed normalization for model output.

```
SIMPLE = T
BITPIX = 8
NAXIS = 3
NAXIS1 = 256
NAXIS2 = 256
NAXIS3 = 256
DATE-OBS= '2026-08-17T00:00:00.000'
TIMESYS = 'UTC'
BSCALE = 0.003937007874016
BZERO = 0.0
BLANK = 255
WCSNAME = 'Heliocentric-cartesian'
CTYPE1 = 'SOLX'
CUNIT1 = 'solRad'
CRPIX1 = 128.5
CRVAL1 = 0.0
CDELT1 = 0.0234375
CTYPE2 = 'SOLY'
CUNIT2 = 'solRad'
CRPIX2 = 128.5
CRVAL2 = 0.0
CDELT2 = 0.0234375
CTYPE3 = 'SOLZ'
CUNIT3 = 'solRad'
CRPIX3 = 128.5
CRVAL3 = 0.0
CDELT3 = 0.0234375
DSUN_OBS= 151470458469.0
CRLN_OBS= 168.635177770
CRLT_OBS= 6.722914954
RSUN_REF= 695700000.0
```

Along each axis, the first and last voxel centers are at -2.98828125 and +2.98828125 solar radii. The outer cell boundaries are at -3 and +3 solar radii.

5.6 Lossless tiled compression

The FITS tiled-image compression convention stores compressed image tiles in a binary table. [GZIP_2](#) is a useful lossless option for 16-bit integer volumes because its byte shuffling often improves compression while preserving the stored integers exactly.

For these integer images, [BLANK](#) also identifies undefined samples in the compressed file, and the reserved values are preserved exactly. No separate mask image is needed.

Some writers quantize floating-point values before compressing them. This step is lossy even when the subsequent compression is lossless, so exact preservation requires quantization to be disabled.

For compatibility with JHV's [nom-tam-fits](#) library, each tile must be one voxel deep along FITS axis 3. In other words, each tile contains part or all of one plane formed by FITS axes 1 and 2 and never spans

several planes. This is a library restriction, not a FITS requirement. A 256 x 256 x 256 volume can use one complete plane per tile:

```
ZCMPTYPE= 'GZIP_2'  
ZTILE1   =          256  
ZTILE2   =          256  
ZTILE3   =           1
```

The physical representation is normally an empty primary HDU followed by a binary-table extension. FITS-aware software can expose the table as the logical 3D image. The table's `BITPIX` and `NAXISi` describe the table storage, and `ZBITPIX` and `ZNAXISi` describe the image it contains. Compression reduces file and transfer size, but not the size of the decompressed array.

6 References

- [FITS Standard 4.0](#), covering headers, sample encoding, WCS, time, and compression
- [FITS tiled-image compression convention](#)
- Astropy documentation: [FITS image data](#) and [WCS](#)
- [SunPy coordinate tools](#)
- W. T. Thompson, [Coordinate systems for solar image data](#), *Astronomy & Astrophysics* 449 (2006), 791-803
- Qorona documentation: [Export to JHelioviewer](#) and [field lines](#)
- J. H. Guo et al., [Modeling the propagation of coronal mass ejections with COCONUT: Implementation of the regularized Biot-Savart law flux rope model](#), *Astronomy & Astrophysics* 683 (2024), A54
- [Khronos glTF Registry](#) and current specification
- [Open Asset Import Library \(Assimp\)](#)
- [VTK vtkGLTFExporter](#)
- [pygltflib](#)

7 Changelog

7.1 Changes in version 0.9

Area	What changed and why
Scope	Set aside experimental volume rendering because its visualization was insufficiently defined. Retained the general WCS and FITS guidance separately for producers exchanging regular-grid data.
SunJSON specification	Added a complete description of the existing SunJSON format, including geometry, appearance, loading, and time selection, to consolidate JHV's heliocentric geometry interfaces in one document.
glTF files and metadata	Clarified <code>.gltf</code> and <code>.glb</code> packaging and gzip support. Tightened the coordinate and solar metadata requirements to make placement and timing unambiguous, including for models whose reference direction is not a physical observer.
glTF appearance	Specified rendering capabilities and limitations, including transparency, lit and unlit materials, and required normals for lit surfaces, so producers can control appearance and anticipate differences from general glTF viewers.
Product preparation and example	Added guidance on careful geometry simplification and decimation to reduce file size, memory use, and rendering cost while preserving relevant structures. Expanded the COCONUT example with boundary points and lit tubes, demonstrating geometry reduction, export, and validation as a starting point for other converters.